

Přírodovědecká fakulta
Univerzita Karlova



Geometrické vyhledávání bodu

Algoritmy počítačové kartografie

Teodor Sorokáč

2. N-GKDPZ
Praha 2026

1 Zadání

Úloha č. 2: Generalizace budov LOD0

Vstup: množina budov $B = \{B_i\}_{i=1}^n$, budova $B_i = \{P_{i,j}\}_{j=1}^m$

Výstup: $G(B_i)$

Ze souboru načtete vstupní data představovaná lomovými body budov a proveďte generalizaci budov do úrovně detailu LOD0. Pro tyto účely použijte vhodnou datovou sadu, např. ZABAGED, testování proveďte nad třemi datovými sadami (historické centrum města, izolovaná zástavba).

Pro každou budovu určete její hlavní směry metodami:

- Minimum Area Enclosing Rectangle
- PCA

U první metody použijte některý z algoritmů pro konstrukci konvexní obálky. Budovu při generalizaci do úrovně LOD0 nahraďte obdélníkem orientovaným v obou hlavních směrech, se středem v těžišti budovy, jeho plocha bude stejná jako plocha budovy. Výsledky generalizace vhodně vizualizujte.

Otestujte a proveďte efektivitu obou metod s využitím hodnotících kritérií. Pokuste se rozhodnout, pro které tvary budo dávají metody nevhodné výsledky, a pro které naopak poskytují vhodnou aproximaci.

Hodnocení:

Krok	Hodnocení
Generalizace budov metodami Minimum Area Enclosing Rectangle a PCA.	15b
<i>Generalizace budov metodou Longest Edge.</i>	<i>+5b</i>
<i>Generalizace budov metodou Wall Average.</i>	<i>+8b</i>
<i>Generalizace budov metodou Weighted Biscetor.</i>	<i>+10b</i>
<i>Implementace další metody konstrukce konvexní obálky.</i>	<i>+5b</i>
<i>Ošetření singulárních případů při generování konvexní obálky.</i>	<i>+2b</i>
<i>Načtení vstupních dat ze *.shp.</i>	<i>+10b</i>
Max celkem:	55b

Řešené bonusové úlohy

V rámci úlohy byly vypracované následující bonusové úlohy: *Generalizace budov metodou Longest Edge (5b)*, *Generalizace budov metodou Wall Average (8b)*, *Generalizace budov metodou Weighted Biscetor (10b)*, *Ošetření singulárních případů při generování konvexní obálky (2b)*, *Načtení vstupních dat ze *.shp (10b)*.

2 Úvod do problematiky úlohy

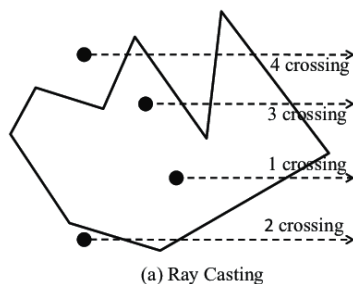
Problém určení polohy bodu, který se nachází vně, uvnitř či na hranici polygonů, jedním ze základních problematik prostorového šetření. Problémem se zabývá především geoinformatika, ale také na něj lze narazit v jiných odvětvích počítačové vědy.

Řešení problému vztahu bodu a mnohoúhelníku lze řešit mnoha způsoby a základním dělením tyto způsoby rozlišujeme na globální a lokální úroveň. Při lokálním řešení se přistupuje pomocí opakování polohy bodu q ve vztahu k mnohoúhelníku. Zjišťujeme zda se bod nachází uvnitř, vně či na hraně mnohoúhelníku. Na globální úrovni se úloha a postup poměrně mění a řešíme polohu bodu q ke všem mnohoúhelníkům z množiny mnohoúhelníků. Výsledkem globální procedury je, že bod se nachází uvnitř jednoho polygonu, vně celé množiny polygonů nebo ležící na hraně či uzlu více mnohoúhelníků.

V rámci řešení lokální procedury jsou pro konvexní mnohoúhelníky nejčastěji využívány dva typy algoritmů. Jedním z nich je paprskový algoritmus (Ray Crossing Algorithm) a druhý Half-plane Test. Pro nekonvexní mnohoúhelníky (jeden úhel je větší 180°) se využívá rovněž algoritmus Ray Crossing a také Winding Number algorithm.

3 Ray Crossing algorithm

Princip algoritmu Ray Crossing pracuje s polopřímkou (paprskem), kterou vede z testovaného bodu libovolným směrem, ale nejčastěji však vodorovně. Po zakreslení polopřímky se počítají průsečíky s hranami polygonu.



Obrázek 1: *Ray Crossing Algorithm* (zdroj: Yan 2012)

Pokud je počet průsečíků lichý, bod se nachází uvnitř polygonu, pokud je sudý, bod se nachází vně polygonu.

$$k(q, P) = \begin{cases} 1, & q \in P, \\ 0 \text{ nebo } 2, & q \notin P. \end{cases}$$

Problémem pro tento algoritmus jsou speciální případy, kde se bod nachází na hraně polygonu či na jeho vrcholu. Pro ošetření těchto speciálních případů je nutné zavést druhou polopřímku, která leží v opačném směru. Dále se postupuje u obou polopřímek stejným způsobem a počítají se průsečíky s hranami polygonu. Pokud se počet průsečíků obou polopřímek nerovná, bod leží na hraně/vrcholu polygonu.

4 Winding Number Algorithm

Tento algoritmus je založen na „ovíjení“ a sčítání úhlů, které svírají vrcholy polygonu k bodu q . Výsledné číslo Winding Number Ω se poté dělí 2π a dává nám tedy údaj o počtu „ovinutí“ okolo bodu q . Pro to, zda se úhly přičtou či odečtou k výslednému Winding Number Ω , se využívá koncept rozdělení na dvě poloroviny a výpočtu determinantu t dvou vektorů: směrového vektoru hrany polygonu $\vec{p}$ a vektoru $\vec{v}$ směřujícího z počátečního vrcholu hrany k bodu q :

$$\begin{aligned}\vec{p} &= (x_{i+1} - x_i, y_{i+1} - y_i) \\ \vec{v} &= (x_q - x_i, y_q - y_i)\end{aligned}$$

Po výpočtu vektorů se spočítá jejich vektorový součin (determinant):

$$t = \det \begin{pmatrix} p_x & p_y \\ v_x & v_y \end{pmatrix}$$

Na základě výsledného determinantu se určí v jaké polorovině bod q leží a zda se úhel bude přičítat či odčítat od výsledného Winding Number. Pokud $t > 0$, bod q leží v levé polorovině a úhel se přičítá, když $t < 0$, bod q náleží pravé polorovině a úhel se odečítá, v případě $t = 0$, bod q leží na přímce a poté platí následující:

$$\exists \omega(p_i, q, p_{i+1}) = \pi$$

Pro výsledné Winding Number platí následující vztah:

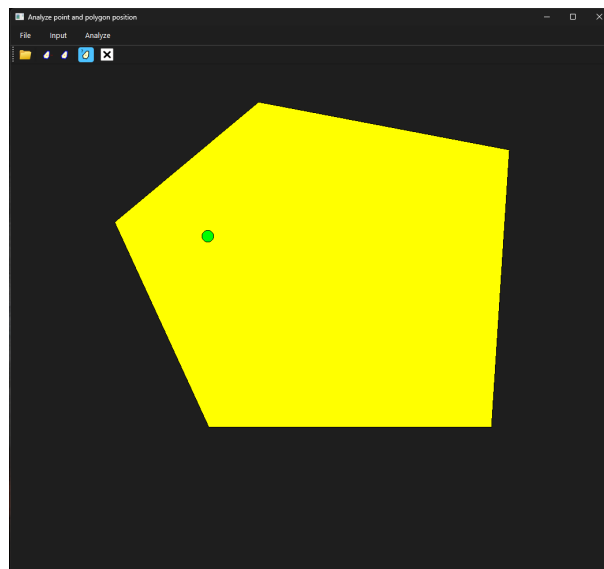
$$\Omega(q, P) = \frac{1}{2\pi} \sum_{i=1}^n \omega(p_i, q, p_{i+1})$$

5 Aplikace

Aplikace se otevře spuštěním skriptu `MainForm.py` a otevře se interaktivní okno, ve kterém lze zakreslit bod a polygon. Nad lištou tlačítek se nachází nabídka tří možností: *File*, *Input* a *Analyze*. Po rozkliknutí záložky *File* se nabídne možnost vložení vlastní polygonové vrstvy z počítače uživatele. Tlačítko *Input* slouží pro přepnutí mezi zakreslením bodu nebo polygonu. Záložka *Analyze* nabízí dvě metody pro vyhledání polohy bodu. Dále je okno doplněno o záložku *Open*, které nabízí možnost vložení souboru ve formátu shapefile, nad kterým lze provést zpracované algoritmy: *Ray Crossing Algorithm* a *Winding Number Algorithm*. Pod hlavní lištou jsou v nabídce tlačítka pro provedení analýzy zpracovaných algoritmů, tlačítko na změnu zakreslení bodu a polygonu a tlačítko pro smazání aktuální vrstvy polygonu a bodu.

V případě vložení vlastní polygonové vrstvy se velikost zobrazované vrstvy přizpůsobí otevřenému oknu aplikace. Po zakreslení bodu, lze provést obě zmiňované metody vyhledávání bodu.

Po spuštění analýzy pomocí libovolné metody se zobrazí informační okno, které uživateli sdělí zda se bod nachází uvnitř, vně či na hranici polygonu. Toto okno lze poté zavřít a bod překreslit na jiné místo a analyzovat jeho polohu znovu. Tlačítkem *OK* se informační okno zavře a pomocí tlačítka *Clear data* se smaže aktuální zakreslený polygon nebo vložená polygonová vrstva.



Obrázek 2: Uživatelské prostředí aplikace

6 Dokumentace

Úloha byla vypracována v softwaru Visual Studio Code v jazyce Python a obsahuje 3 soubory se skripty, které na sebe odkazují (algorithms.py, draw.py, MainForm.py). Pro zapisování skriptů byl použit formát CamelCase.

algorithms.py

V tomto souboru byly uloženy následující funkce:

- **def pointOnVertex** - Funkce zjišťuje zda se bod q nachází na vrcholu polygonu P , pokud se souřadnice x a y shodují, bod leží na vrcholu polygonu.
- **def rayCrossing** - Funkce analyzuje polohu bodu vůči polygonu metodou Ray Crossing Algorithm. Funkce prochází pomocí cyklu `for` každý vrchol polygonu. V případě že se bod nachází vně polygonu algoritmus vrací hodnotu 0, pokud se bod nachází uvnitř polygonu algoritmus vrací hodnotu 1 a v případě, že bod leží na vrcholu či hraně polygonu vrací algoritmus hodnotu -1.
- **def calculateAngle** - Tato funkce vypočítává velikost vnitřního úhlu mezi vektorem $v1$ a $v2$. Pro výpočet úhlu nejprve vypočítá vektory vycházející z bodu q do bodů $P1$ a $P2$. V dalším kroce spočítá jejich délku a poté spočítá úhel mezi vektory.
- **def getPointLocation** - Funkce analyzuje polohu bodu a přímky definovanou body $P1$ a $P2$ výpočtem determinantu. Pokud $\det = 0$, bod leží na přímce, v případě když $\det > 0$, bod leží vlevo od přímky, v případě $\det < 0$, bod leží vpravo od přímky.
- **def windingNumber** - Funkce analyzuje polohu bodu vůči polygonu metodou Winding Number Algorithm. Pomocí cyklu `while` prochází všechny přímky (spojnice mezi vrcholy polygonu). V případě že se bod nachází uvnitř polygonu vrací hodnotu 1, pokud se bod nachází vně polygonu vrací hodnotu 0 a pokud se bod nachází na vrcholu či hraně vrací hodnotu -1.
- **def inMinMaxBox** - Funkce kontroluje zda se bod nachází v min-max boxu polygonu. V případě, že se bod nenachází v min-max boxu daného polygonu nemusí se provádět výpočet algoritmu pro vyhodnocení polohy bodu vůči polygonu, protože s jistotou v daném polygonu neleží.

draw.py

V tomto souboru byly uloženy následující funkce:

- **def mousePressEvent** - Funkce určuje souřadnice zakreslení bodu nebo vrcholu polygonu, na základě kliknutí do plátna v aplikaci.

- **def changeStatus** - Funkce přepíná mezi kreslením polygonu nebo bodu.
- **def paintEvent** - Funkce vykreslí polygon, polygonovou vrstvu či bod do plátna.
- **def clearData** - Funkce vymaže vykreslená data.
- **def getQ** - Funkce vrací analyzovaný bod.
- **def getPol** - Funkce vrací vykreslený polygon.
- **def getPolygons** - Funkce vrací polygony jako seznam, pokud se jedná o vrstvu s více polygony.
- **def setHighlightedPolygons** - Funkce nahradí seznam zvýrazněných polygonů na nový seznam indexů.
- **def paintRes** - Funkce přidá vybraný polygon do seznamu zvýrazněných polygonů.
- **def clearRes** - Funkce vymaže vykreslený polygon nebo vykreslenou polygonovou vrstvu a bod.
- **def openFile** - Funkce otevře systémové okno, ve kterém je možnost nahrání vlastní polygonové vrstvy ve formátu *.shp.
- **def loadSHP** - Funkce vykreslí zvolenou polygonovou vrstvu formátu *.shp do aplikace a polygony zařadí do seznamu. Funkce zajišťuje zpracování vstupní geometrie a upraví ji na souřadnice dle velikosti plátna aplikace.

MainForm.py

V tomto souboru byly uloženy následující funkce:

- **def openFileClick** - Funkce zavolá funkci openFile.
- **def changeStatusClick** - Funkce zavolá funkci changeStatus.
- **def clearClick** - Funkce zavolá funkci clearData.
- **def getRes** - Funkce zavolá funkci getQ a skript algorithms.py. Funkce zajišťuje průběh algoritmů nad analyzovaným bodem a vstupním polygonem nebo vstupní polygonovou vrstvou. Nejdříve proběhne funkce inMinMaxBox a v případě, že bod náleží min-max boxu polygonu postupuje dále a na základě zvoleného algoritmu analyzuje polohu bodu k polygonu.
- **def analyzePointAndPositionClick** - zavolá funkci getRes pro zvolený algoritmus Ray Crossing.
- **def analyzePointAndPositionWNClick** - zavolá funkci getRes pro zvolený algoritmus Winding Number.

Seznam literatury

BAYER, Tomáš. *Algoritmy v digitální kartografii*. Praha: Karolinum, 2008. ISBN 978-80-246-1499-1.

BAYER, T. (2026): Point Location Problem. Presentace pro přednášku předmětu Algoritmu počítačové kartografie, Katedra aplikované geoinformatiky a kartografie, Přírodovědecká a fakulta UK [cit. 22. 3. 2026].

AN, D., ZHAO, Z., NG, W. K. (2012): Monochromatic and Bichromatic Reverse Nearest Neighbor Queries on Land Surfaces. 10.1145/2396761.2396880.